

CST8509 Lab5 Docker

In this lab, you will explore [Docker](#), a powerful tool for building, deploying, and managing software in isolated environments called **containers**. Unlike traditional virtual machines, containers share the host's operating system kernel, making them lightweight, fast to start, and resource-efficient.

Why Use Docker for ROS 2 and Reinforcement Learning?

We have seen that developing with the **Robot Operating System (ROS 2)** often involves complex dependencies and strict version requirements. Docker is particularly beneficial for robotics students because it:

- **Ensures Consistency:** It eliminates "it works on my machine" issues by bundling the application with its exact dependencies, configurations, and OS.
- **Prevents Dependency Conflicts:** You can run different ROS 2 versions (e.g., Humble on Ubuntu 22.04) on the same host machine without polluting your primary system or causing package conflicts.
- **Simplifies Deployment:** Docker makes your project portable, allowing it to run identically on a laptop, a physical robot, or a cloud server.
- **Enables Reproducibility:** Students and researchers can share an identical environment, ensuring that code that compiles for one person will compile for everyone else.

Lab Objective

The goal of this lab is to use AI to help you **dockerize your ROS 2 Assignment 2 project** targeting the **Ubuntu Jammy (22.04)** and **ROS 2 Humble** distribution. You will:

1. **Craft a Dockerfile:** Create a blueprint that pulls an appropriate base image and installs necessary system dependencies.
2. **Build a Custom Image:** Use the `docker build` command to generate a ready-to-run environment containing your specific robotics code.
3. **Launch Containers:** Use `docker run` or **Docker Compose** to access a shell prompt and start your ROS 2 nodes, managing networking and graphical interfaces (like RViz) so they can communicate with your host machine.

By the end of this session, you will have a robust, repeatable development environment that simplifies the transition from writing code to deploying it on real hardware.

Instructions

The primary development platform for this lab is one of the following:

- Your Ubuntu 22.04 loaner laptop with docker and docker-compose installed
- Your Windows laptop with WSL2 and Docker Desktop installed and configured for WSL2

You are allowed to use AI (see below a suggested Prompt Template that you need to modify).

Deliverables

Submit the following items to Brightspace:

- Dockerfile and docker-compose.yml
- Actual Prompts you used to complete the dockerization:
 - Filled out template prompt (or your own prompt), and the response
 - Followup prompts, and responses

Demonstration

Show your lab instructor that you are able to run the docker container, access a bash prompt, and launch your Assignment 2 project.

Suggested Prompt Template

I need you to generate a complete Dockerfile and docker-compose.yml for my ROS 2 Humble project.

 PROJECT CONTEXT

I am completing an assignment that uses:

- ROS 2 Humble on Ubuntu 22.04
- Gazebo Classic
- RViz2

- Create3 simulator modified to include:
 - A camera added to the robot URDF
 - A moving red ball actor in the AWS small house world
 - A patched AWS small house world
- A custom Gymnasium-based RL node
- A Python virtual environment with common ML dependencies
- My own ROS 2 workspace containing:
<INSERT YOUR PACKAGE NAMES HERE>

✅ WHAT I NEED FROM YOU

I want you to create:

(1) A Dockerfile that:

- Uses ROS 2 Humble (FROM osrf/ros:humble-desktop)
- Supports building a full ROS 2 workspace using colcon
- Installs Python 3 venv and ML packages (gymnasium, stable-baselines3, etc.)
- Installs Gazebo Classic and required Mesa/OpenGL libs
- Copies my ROS 2 workspace into /ros2_ws/src
- Applies my patches (URDF camera, world modifications, actors)
- Creates a Python virtual environment in /opt/ros_venv and activates it
- Builds all my packages with colcon build
- Sources /opt/ros/humble/setup.bash, /ros2_ws/install/setup.bash,
and the virtual environment
- Supports launching Gazebo + RViz2 from inside the container

(2) A docker-compose.yml file that:

- Builds the container with ARG ROS_DISTRO=humble
- Uses host networking (required for ROS 2 DDS discovery)
- Exposes GUI applications by mapping:
 /tmp/.X11-unix:/tmp/.X11-unix
- Passes through environment variables:
 DISPLAY, RMW_IMPLEMENTATION, IGNITION_VERSION=fortress
- Enables optional GPU acceleration using:
 NVIDIA_VISIBLE_DEVICES=all
 NVIDIA_DRIVER_CAPABILITIES=all
- Runs the container in privileged mode for device access
- Provides a command override that launches a bash shell
- (Optional) Mounts volumes for debugging if you think they are useful

EXTRA REQUIREMENTS

Please ensure:

- The Dockerfile is multi-stage (clone → build)
- My workspace lives in /ros2_ws
- The final image drops into bash by default
- The Compose file is fully valid YAML (spaces, not tabs)

PATCH LOCATIONS

My patches live here:

<INSERT PATHS TO PATCH FILES HERE>

REPOSITORIES TO CLONE (IF ANY)

<LIST YOUR REPOS HERE OR SAY “NONE”>

WHAT TO OUTPUT

Please output:

1. The complete Dockerfile
2. The complete docker-compose.yml
3. Any additional instructions I need to run:

docker compose build

docker compose run <service>

ros2 launch <my package> <my launch file>

Make sure the output is clean, accurate, and ready to use.

Be explicit with paths and commands.